 # implement training loop of univariate logistic regression with L2 regularization in numpy

(let: m = no of samples, epochs = no of iterations, w0/w1 = weights, X = feature, y = binary output, lr = learning rate, lam = regularization parameter)

```
for i in range(epochs):
    z = w0 + (w1 * X)
    ypred = 1 / (1 + np.exp(-z))
    loss = (-1 / m) * np.sum(y * np.log(ypred) + (1 - y) * np.log(1 - ypred)) + (lam / (2 * m)) * (w1 ** 2)
    dw0 = (1 / m) * np.sum(ypred - y)
    dw1 = (1 / m) * np.sum((ypred - y) * X) + (lam / m) * w1
    w0 = w0 - (lr * dw0)
    w1 = w1 - (lr * dw1)
```

(Note: In standard practice, the bias weight w0 is not regularized, so the L2 penalty is only applied to w1 in the loss and gradient calculations.)

Logistic Regression with L2 Regularization

Hypothesis Function

sigmoid(z) where $z = w0 + w1(x1^2) + w2(x2^3)$

Dataset

Sample 1: X1 = 2, X2 = -4, y = 1
Sample 2: X1 = -2, X2 = 4, y = 0

Initial Parameters

Weights: w0 = 1, w1 = 2, w2 = 1
Learning Rate: alpha = 0.1
Regularization Parameter: lambda = 1
(Note: In standard practice, the bias term (w0) is not regularized.)

Tasks

1. Apply max normalization to the features X1 and X2

(divide each feature column by its maximum absolute value)

absolute max for X1: max(|2|, |-2|) = 2
absolute max for X2: max(|-4|, |4|) = 4
Normalized Sample 1: x1 = 2 / 2 = 1, x2 = -4 / 4 = -1
Normalized Sample 2: x1 = -2 / 2 = -1, x2 = 4 / 4 = 1

2. Perform Iteration 1

(calculate linear output z, calculate predictions using sigmoid, calculate L2 Regularized Log Loss, calculate regularized gradients for weights, update the weights)

Linear Output (z) and Predictions:

Sample 1: $z = 1 + 2(1)^2 + 1(-1)^3 = 2$
Prediction (y hat) = $1 / (1 + e^{-2}) = 0.8808$
Sample 2: $z = 1 + 2(-1)^2 + 1(1)^3 = 4$
Prediction (y hat) = $1 / (1 + e^{-4}) = 0.9820$

Cost Function (Log Loss with L2 Regularization):

= [(-1 / m) * Sum of (y * ln(y hat) + (1 - y) * ln(1 - y hat))] + [(lambda / 2m) * $(w1^2 + w2^2)$]
= [(-1 / 2) * [(1 * ln(0.8808) + 0) + (0 + 1 * ln(1 - 0.9820))]] + [(1 / 4) * $(2^2 + 1^2)$]
= 2.0722 + 1.2500 = 3.3222

Gradients:

Gradient for w0 (unregularized):

= (1 / m) * Sum of (predicted - actual)
= (1 / 2) * [(0.8808 - 1) + (0.9820 - 0)] = 0.4314

Gradient for w1 (regularized):

= (1 / m) * Sum of [(predicted - actual) * $x1^2$] + (lambda / m) * w1
= 0.4314 + (1 / 2) * 2 = 0.4314 + 1.0000 = 1.4314

Gradient for w2 (regularized):

= (1 / m) * Sum of [(predicted - actual) * $x2^3$] + (lambda / m) * w2
= (1 / 2) * [$(-0.1192)(-1)^3 + (0.9820)(1)^3$] + (1 / 2) * 1 = 0.5506 + 0.5000 = 1.0506

Weight Updates:

w0 = w0 - alpha * dw0 = 1 - 0.1(0.4314) = 0.9569
w1 = w1 - alpha * dw1 = 2 - 0.1(1.4314) = 1.8569

w2 = w2 - alpha * dw2 = 1 - 0.1(1.0506) = 0.8949

3. Perform Iteration 2

(calculate linear output z, calculate predictions using sigmoid, calculate L2 Regularized Log Loss, calculate regularized gradients for weights, update the weights)

Linear Output (z) and Predictions:

Sample 1: $z = 0.9569 + 1.8569(1)^2 + 0.8949(-1)^3 = 1.9189$

Prediction (y hat) = $1 / (1 + e^{-1.9189}) = 0.8720$

Sample 2: $z = 0.9569 + 1.8569(-1)^2 + 0.8949(1)^3 = 3.7087$

Prediction (y hat) = $1 / (1 + e^{-3.7087}) = 0.9761$

Cost Function (Log Loss with L2 Regularization):

= [(-1 / m) * Sum of (y * ln(y hat) + (1 - y) * ln(1 - y hat))] + [(lambda / 2m) * ($w1^2 + w2^2$)]

= [(-1 / 2) * [(1 * ln(0.8720) + 0) + (0 + 1 * ln(1 - 0.9761))]] + [(1 / 4) * ($1.8569^2 + 0.8949^2$)]

= 1.9355 + 1.0622 = 2.9977

Gradients:

Gradient for w0 (unregularized):

= (1 / m) * Sum of (predicted - actual)

= (1 / 2) * [(0.8720 - 1) + (0.9761 - 0)] = (1 / 2) * [-0.1280 + 0.9761] = 0.4241

Gradient for w1 (regularized):

= (1 / m) * Sum of [(predicted - actual) * $x1^2$] + (lambda / m) * w1

= (1 / 2) * [$(-0.1280)(1)^2 + (0.9761)(-1)^2$] + (1 / 2) * 1.8569 = 0.4241 + 0.9285 = 1.3526

Gradient for w2 (regularized):

= (1 / m) * Sum of [(predicted - actual) * $x2^3$] + (lambda / m) * w2

= (1 / 2) * [$(-0.1280)(-1)^3 + (0.9761)(1)^3$] + (1 / 2) * 0.8949 = 0.5521 + 0.4475 = 0.9996

Weight Updates:

w0 = w0 - alpha * dw0 = 0.9569 - 0.1(0.4241) = 0.9145

w1 = w1 - alpha * dw1 = 1.8569 - 0.1(1.3526) = 1.7216

w2 = w2 - alpha * dw2 = 0.8949 - 0.1(0.9996) = 0.7949